CENTRO UNIVERSITÁRIO DE BRASÍLIA — UNICEUB
Curso de Análise e Desenvolvimento de Sistemas
Disciplina: Bootcamp III

Eduardo Costa Alberto de Sousa
RA 22354024

SYSTEM DESIGN DOCUMENT (SDD)

TaskAnalyzer — documento de projeto do sistema

Arquitetura, contrato, decisões de projeto e infraestrutura de entrega

Versão 1.0 — 20/10/2026 (Fase 3)
Documento derivado de `specs/task_analyzer_spec.md`, que é a fonte da verdade do contrato

Brasília
Outubro de 2026

1. Propósito e escopo

Este documento descreve o projeto do TaskAnalyzer: componentes, contrato de dados, regras de validação, decisões de projeto e a infraestrutura de integração contínua e de contêiner. Ele **não cria regras novas**: o contrato de negócio está em `specs/task_analyzer_spec.md` (fonte da verdade) e, em formato estruturado e rastreável, em `specs/requirements.yaml`. Em caso de divergência, prevalece a especificação em Markdown.

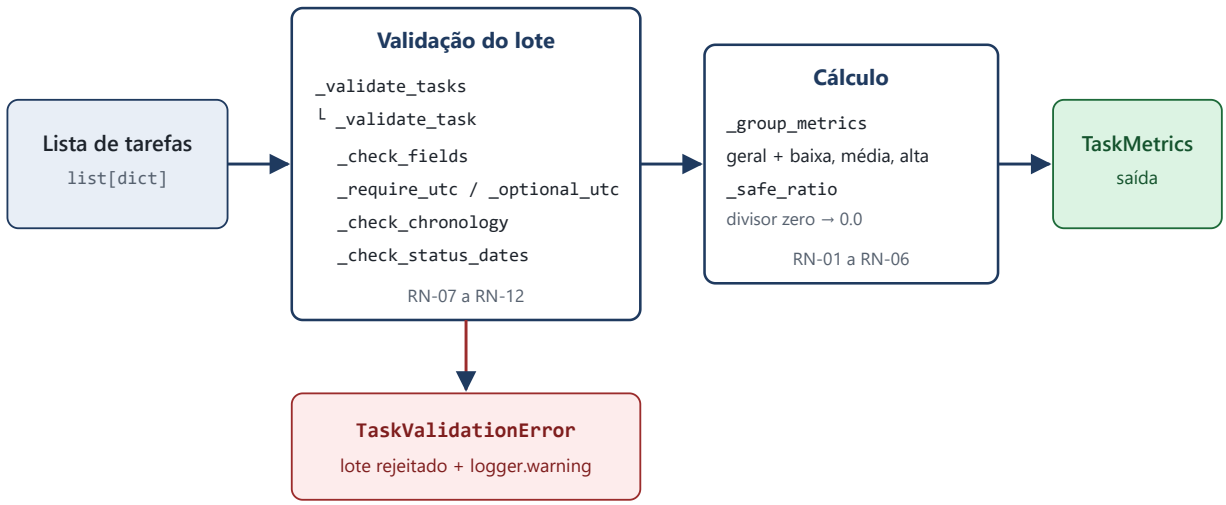
O TaskAnalyzer analisa uma lista de tarefas em memória e calcula indicadores de produtividade — totais, tempo médio de conclusão e taxa de atraso —, gerais e por prioridade. Entradas que violam o contrato geram `TaskValidationError`. Fora do escopo: persistência, interface gráfica, API de rede e dependências além do `pytest`.

> **Origem do contrato.** A Fase 1 não foi entregue. A especificação foi reconstruída a partir dos enunciados das Fases 1 e 2 e homologada na Fase 2 (Pull Request #1).

2. Visão geral da arquitetura

O sistema é um único módulo Python (`src/task_analyzer.py`) com uma função pública. O processamento ocorre em duas etapas: **validação** de todo o lote e **cálculo** dos indicadores. Se qualquer tarefa for inválida, o lote inteiro é rejeitado antes do cálculo.

Figura 1 — Fluxo de processamento de `analyze_tasks`



Fonte: elaborado pelo autor a partir de `src/task_analyzer.py`.

3. Componentes

Tabela 1 — Componentes de `src/task_analyzer.py`

Componente	Responsabilidade	Regras
`analyze_tasks` (pública)	Orquestra validação e cálculo; registra log de sucesso ou de rejeição.	Interface
`TaskValidationError`	Exceção específica (subclasse de `ValueError`) para violações do contrato.	RN-07 a RN-12
`GroupMetrics`, `TaskMetrics`	`TypedDict` que tipam a saída de um grupo e a saída completa.	Saída
`_validate_tasks`	Exige lista; valida cada tarefa e a unicidade dos IDs.	RN-10, RN-12
`_validate_task`	Valida uma tarefa e devolve cópia com datas normalizadas em UTC.	RN-07 a RN-11
`_check_fields`	Campos obrigatórios, `id_tarefa`, prioridade e status.	RN-10
`_require_utc`, `_optional_utc`	Exigem `datetime` com fuso UTC.	RN-08
`_check_chronology`	Rejeita início/conclusão antes da criação e conclusão antes do início.	RN-07
`_check_status_dates`	Coerência entre status e datas de execução.	RN-09
`_group_metrics`	Totais, médias (min/h) e taxa de atraso de um grupo, com arredondamento.	RN-01 a RN-06
`_safe_ratio`	Divisão que retorna `0.0` quando o divisor é zero.	RN-05

4. Contrato de dados

Tabela 2 — Entrada (cada tarefa)

Campo	Tipo	Obrigatório	Regra
`id_tarefa`	int	Sim	Inteiro positivo e único
`data_criacao`	datetime	Sim	Fuso UTC
`data_inicio`	datetime ou None	Não	Fuso UTC quando existente
`data_conclusao`	datetime ou None	Não	Fuso UTC quando existente
`prazo`	datetime	Sim	Fuso UTC
`prioridade`	str	Sim	`baixa`, `média`, `alta`
`status`	str	Sim	`concluída`, `pendente`, `cancelada`

Tabela 3 — Saída (`TaskMetrics`; cada prioridade repete os oito campos de grupo)

Campo	Tipo	Descrição
`total_tarefas`, `quantidade_tarefas`	int	Tarefas do grupo (nomes da Fase 2 e da Fase 1)
`total_concluidas`, `total_pendentes`, `total_canceladas`	int	Contagem por status
`tempo_medio_conclusao_min`, `tempo_medio_conclusao_horas`	float	Média de `data_conclusao − data_inicio` das concluídas
`taxa_atraso_percentual`	float	% de concluídas com `data_conclusao > prazo`
`indicadores_por_prioridade`	dict	Chaves `baixa`, `média`, `alta`, sempre presentes

5. Regras de negócio e tratamento de erros

As regras RN-01 a RN-12 estão detalhadas na especificação e em `specs/requirements.yaml`, cada uma com os testes que a verificam. Em resumo:

- **Cálculo (RN-01 a RN-06):** só tarefas concluídas entram na média e na taxa de atraso; conclusão no prazo não é atraso; sem concluídas, médias e taxa valem `0.0`; tempos e percentuais com duas casas decimais.
- **Validação (RN-07 a RN-10):** cronologia impossível, datas fora de UTC, incoerência entre status e datas, campos ausentes, IDs inválidos ou repetidos e valores fora das enumerações geram `TaskValidationError`.
- **Erros (RN-11 e RN-12):** mensagens no formato `Tarefa <id>: <motivo>` (ex.: `Tarefa 1: data_conclusao anterior à data_criacao`); uma tarefa inválida rejeita o lote inteiro, sem resultado parcial.

Logging: o módulo usa `logging.getLogger(__name__)`. Registra `INFO` com a quantidade de tarefas analisadas e `WARNING` com o motivo da rejeição; não registra o conteúdo das tarefas. A configuração de handlers fica a cargo de quem usa o módulo.

6. Decisões de projeto

Tabela 4 — Decisões registradas

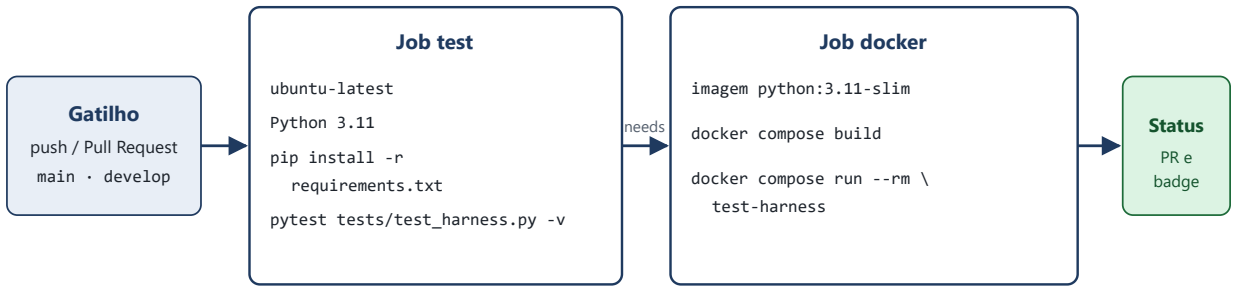
Decisão	Alternativa descartada	Justificativa
Datas inconsistentes lançam `TaskValidationError` e rejeitam o lote	Ignorar a tarefa (regras da Fase 1, p. 8)	A Fase 2 exige a exceção; ignorar em silêncio distorceria as métricas sem aviso.
Sem tarefas concluídas, retornar `0.0`	Lançar exceção (Cenário 2 da Fase 1)	Os casos de borda da Fase 2 exigem `0.0` sem exceção.
Saída com minutos e horas; `quantidade_tarefas` e `total_tarefas`	Escolher um só nome ou unidade	Compatibilidade com os dois enunciados.
Rejeitar datas sem fuso	Assumir UTC implicitamente	Regra “fuso sempre UTC”; evita comparar datas com e sem fuso (`TypeError`).
`pytest==9.1.1` fixado	Faixa de versões	Mesma dependência no ambiente local, no CI e na imagem Docker.
Contêiner executa como usuário sem root	Executar como root	Princípio do menor privilégio.
Actions fixadas por SHA de commit	Tags móveis (`@v7`)	Evita que código de terceiros mude sem revisão.

7. Infraestrutura de entrega

7.1 Integração contínua (GitHub Actions)

O workflow `.github/workflows/ci.yml` é acionado por *push* e *Pull Request* nas branches `main` e `develop` (e manualmente, por `workflow_dispatch`). Tem permissão mínima (`contents: read`) e dois jobs em sequência:

Figura 2 — Pipeline de CI



Fonte: elaborado pelo autor a partir de `.github/workflows/ci.yml` .

7.2 Contêiner (Docker)

O `Dockerfile` parte de `python:3.11-slim`, instala as dependências de `requirements.txt` numa camada própria (reaproveitada em cache), copia apenas `pytest.ini` , `src/` e `tests/` e define

`pytest tests/test_harness.py -v` como comando principal (`CMD`). O `docker-compose.yml` declara o serviço `test-harness`:

```
docker compose run --rm test-harness              # Compose
docker build -t sdd-task-analyzer . && docker run --rm sdd-task-analyzer   # Docker puro
```

O `.dockerignore` exclui `.git`, ambientes virtuais e caches do contexto de build.

8. Estratégia de qualidade

- **Test Harness** (`tests/test_harness.py`): 16 testes, 21 casos com parametrização, cobrindo os Cenários 1 e 2, os casos de borda e as validações do contrato.
- **Execução automatizada:** a cada push ou PR em `main` / `develop`, em Python 3.11 e dentro da imagem Docker.
- **Teste de mutação manual** (auditoria da Fase 2): 10 erros introduzidos no código, 10 detectados.
- **Verificações estáticas** (auditoria da Fase 2, fora do CI): `mypy --strict` e `ruff` (PEP 8, docstrings Google).

9. Restrições de governança

O código é gerado e revisado sob as regras de `CONTEXT_RULES.md`: Python 3.11+, type hints, docstrings Google, exceções específicas, nenhuma biblioteca externa além do `pytest`, proibição de afrouxar testes, de persistir dados e de assumir comportamentos fora do contrato. Toda alteração entra na `main` por Pull Request, com CI verde e homologação humana.